

EXERCÍCIOS PRÁTICOS

Reconhecimento de gestos e expressões com MediaPipe

Python • OpenCV • MediaPipe

Material didático com código-base e propostas de investigação

Apresentação

Esta lista propõe cinco atividades práticas de visão computacional com Python e MediaPipe. Os exercícios trabalham a detecção de marcos (landmarks) da mão e do rosto e a criação de regras geométricas simples para classificar gestos e estados faciais.

O código-base a seguir reúne classificações iniciais de mão aberta/fechada, contagem de dedos, polegar para cima/baixo, boca aberta/fechada e direção aproximada do rosto. As regras são heurísticas para fins didáticos: devem ser testadas e calibradas, pois podem falhar conforme a pose, a iluminação, a distância e as diferenças individuais.

Objetivos de aprendizagem

- Acessar a webcam e processar quadros com OpenCV.
- Interpretar landmarks normalizados produzidos pelo MediaPipe.
- Construir classificadores simples a partir de distâncias e relações geométricas.
- Testar limites, registrar erros e discutir as limitações de uma solução baseada em regras.

Preparação do ambiente

Instale as dependências em um ambiente Python:

```
pip install mediapipe opencv-python
```

O exemplo usa a interface `mp.solutions`, disponível em determinadas versões do MediaPipe. Se sua instalação não oferecer esse módulo, consulte a documentação da versão instalada e adapte o exemplo para a API MediaPipe Tasks. Para encerrar a janela, pressione Esc.

Código-base

Salve o código em um arquivo Python e execute-o em um ambiente com acesso à webcam. O programa espelha a imagem para facilitar o uso como espelho, desenha os landmarks da mão e apresenta as classificações disponíveis.

```
import cv2
import mediapipe as mp
import math

mp_hands = mp.solutions.hands
mp_face_mesh = mp.solutions.face_mesh
mp_drawing = mp.solutions.drawing_utils

def distancia(a, b):
    """Distância euclidiana entre dois pontos normalizados (x, y)."""
    return math.hypot(a.x - b.x, a.y - b.y)

def contar_dedos(hand):
    """Conta dedos estendidos em uma pose aproximadamente frontal."""
    lm = hand.landmark
    quantidade = 0

    # Indicador, médio, anelar e mínimo: ponta acima da articulação.
    for ponta, articulacao in [(8, 6), (12, 10), (16, 14), (20, 18)]:
        if lm[ponta].y < lm[articulacao].y:
            quantidade += 1

    # Regra inicial para o polegar, baseada na distância entre landmarks.
    ponta_polegar = distancia(lm[4], lm[5])
    articulacao_polegar = distancia(lm[3], lm[5])
    if ponta_polegar > articulacao_polegar * 1.15:
        quantidade += 1

    return quantidade
```

```

def classificar_mao(hand):
    quantidade = contar_dedos(hand)
    if quantidade >= 4:
        return "Mao aberta"
    if quantidade <= 1:
        return "Mao fechada"
    return f"Mao parcialmente aberta ({quantidade} dedos)"

def classificar_polegar(hand):
    lm = hand.landmark
    outros_estendidos = sum(
        lm[ponta].y < lm[articulacao].y
        for ponta, articulacao in [(8, 6), (12, 10), (16, 14), (20, 18)]
    )
    if outros_estendidos > 1:
        return "Outro gesto"

    diferenca = lm[4].y - lm[3].y
    if diferenca < -0.04:
        return "Polegar para cima"
    if diferenca > 0.04:
        return "Polegar para baixo"
    return "Polegar indefinido"

def classificar_boca(face):
    lm = face.landmark
    # 13 e 14: lábios internos; 61 e 291: cantos da boca.
    altura_boca = distancia(lm[13], lm[14])
    largura_boca = distancia(lm[61], lm[291])
    if largura_boca < 1e-6:
        return "Boca indefinida"

    proporcao = altura_boca / largura_boca
    return "Boca aberta" if proporcao > 0.15 else "Boca fechada"

def classificar_direcao_rosto(face):
    lm = face.landmark
    olho_esquerdo = lm[33]
    olho_direito = lm[263]
    nariz = lm[1]
    centro_olhos_x = (olho_esquerdo.x + olho_direito.x) / 2
    distancia_olhos = abs(olho_direito.x - olho_esquerdo.x)
    if distancia_olhos < 1e-6:
        return "Direcao indefinida"

    desvio = (nariz.x - centro_olhos_x) / distancia_olhos
    if desvio < -0.12:
        return "Rosto virado para um lado"
    if desvio > 0.12:
        return "Rosto virado para o outro lado"
    return "Rosto aproximadamente de frente"

camera = cv2.VideoCapture(0)
if not camera.isOpened():
    raise RuntimeError("Nao foi possivel acessar a webcam.")

with mp_hands.Hands(
    max_num_hands=2,
    min_detection_confidence=0.6,
    min_tracking_confidence=0.5

```

```

) as detector_maos, mp_face_mesh.FaceMesh(
    max_num_faces=1,
    min_detection_confidence=0.6,
    min_tracking_confidence=0.5
) as detector_rosto:

while camera.isOpened():
    sucesso, imagem = camera.read()
    if not sucesso:
        break

    imagem = cv2.flip(imagem, 1)
    imagem_rgb = cv2.cvtColor(imagem, cv2.COLOR_BGR2RGB)
    resultado_maos = detector_maos.process(imagem_rgb)
    resultado_rosto = detector_rosto.process(imagem_rgb)

    if resultado_maos.multi_hand_landmarks:
        for hand in resultado_maos.multi_hand_landmarks:
            mp_drawing.draw_landmarks(
                imagem, hand, mp_hands.HAND_CONNECTIONS
            )
            cv2.putText(imagem, classificar_mao(hand), (20, 40),
                cv2.FONT_HERSHEY_SIMPLEX, 0.7, (0, 255, 0), 2)
            cv2.putText(imagem, classificar_polegar(hand), (20, 75),
                cv2.FONT_HERSHEY_SIMPLEX, 0.65, (0, 200, 255), 2)
            cv2.putText(imagem, f"Dedos: {contar_dedos(hand)}", (20, 110),
                cv2.FONT_HERSHEY_SIMPLEX, 0.65, (255, 200, 0), 2)

    if resultado_rosto.multi_face_landmarks:
        for face in resultado_rosto.multi_face_landmarks:
            cv2.putText(imagem, classificar_boca(face), (20, 150),
                cv2.FONT_HERSHEY_SIMPLEX, 0.65, (255, 0, 255), 2)
            cv2.putText(imagem, classificar_direcao_rosto(face), (20, 185),
                cv2.FONT_HERSHEY_SIMPLEX, 0.65, (255, 255, 0), 2)

    cv2.imshow("Gestos e expressoes", imagem)
    if cv2.waitKey(1) & 0xFF == 27:
        break

camera.release()
cv2.destroyAllWindows()

```

Exercícios

1. Mão aberta e mão fechada

Objetivo: classificar a mão em três estados: aberta, fechada ou parcialmente aberta. Use a função `classificar_mao()` do código-base e compare sua resposta com a quantidade de dedos calculada.

Tarefas:

- Teste a classificação com a palma voltada para a câmera, com a mão inclinada e com um ou mais dedos parcialmente dobrados.
- Ajuste os limites usados para separar mão aberta, fechada e posições intermediárias.
- Exiba “mão não detectada” quando nenhum landmark estiver disponível, sem reutilizar a classificação do quadro anterior.

Questão para análise: em quais condições a contagem de dedos deixa de representar adequadamente o estado aberto ou fechado da mão?

2. Contagem da quantidade de dedos

Objetivo: exibir continuamente um número de 0 a 5 para cada mão detectada. A função `contar_dedos()` utiliza uma comparação vertical para os quatro dedos e uma regra de distância para o polegar.

Tarefas:

- Teste as cinco posições de contagem (zero a cinco) com a palma aproximadamente voltada para a webcam.
- Registre pelo menos três casos de contagem incorreta e descreva a pose em que ocorreram.
- Investigue como a orientação da mão, a oclusão dos dedos e o espelhamento da imagem afetam as regras.

Extensão: acrescente uma pequena janela de tolerância ou suavização temporal para reduzir oscilações entre números em quadros consecutivos.

3. Polegar para cima e para baixo

Objetivo: identificar os gestos “polegar para cima” e “polegar para baixo” e rejeitar outras configurações. A função `classificar_polegar()` verifica a posição vertical do polegar e se os demais dedos parecem recolhidos.

Tarefas:

- Experimente com as duas mãos e com o polegar inclinado para os lados.
- Calibre a margem vertical (0.04 no exemplo) para evitar decisões instáveis próximas do limite.
- Mostre “outro gesto” se os demais dedos estiverem estendidos ou se a orientação não permitir uma classificação segura.

Questão para análise: por que o significado de “esquerda” e “direita” pode mudar quando a imagem é espelhada?

4. Boca aberta e boca fechada

Objetivo: estimar se a boca está aberta comparando a distância vertical entre os lábios com a largura da boca. A razão entre essas medidas reduz, mas não elimina, a influência do tamanho do rosto na imagem.

Tarefas:

- Mostre na imagem a proporção altura/largura calculada.
- Colete observações com a boca fechada, ligeiramente aberta e bem aberta; escolha e justifique um limite adequado.
- Teste fala, sorriso, rosto inclinado e diferentes distâncias da câmera; identifique falsos positivos e falsos negativos.

Atenção: o valor 0.15 do exemplo é apenas um ponto de partida, não um limiar universal.

5. Direção do rosto

Objetivo: reconhecer se o rosto está aproximadamente de frente ou virado para um dos lados. A regra inicial compara a posição horizontal do nariz com o centro entre os olhos.

Tarefas:

- Calibre a classificação olhando para frente, para a esquerda e para a direita; registre a saída e a direção percebida pelo participante.
- Verifique se o espelhamento da webcam inverte os rótulos e torne as mensagens explícitas em relação à imagem exibida.
- Inclua um estado indefinido quando os landmarks estiverem ausentes ou a estimativa ficar próxima do limite.

Extensão: investigue uma estimativa de pose facial que utilize vários landmarks para estimar yaw, pitch e roll, em vez de depender apenas do deslocamento do nariz.

Avaliação e cuidados

Protocolo de testes

- Teste com mais de uma pessoa, diferentes distâncias e variações de iluminação.
- Separe os exemplos usados para calibrar os limiares dos exemplos usados para avaliar o comportamento final.
- Registre acertos, erros e situações em que o sistema deve responder “indefinido”.
- Apresente imagens ou uma tabela com as condições testadas e os resultados.

Limitações e privacidade

As regras geométricas são aproximações e podem não funcionar para todas as pessoas ou poses. A webcam pode não estar disponível, os landmarks podem falhar por oclusão ou iluminação insuficiente e os limiares precisam de validação. Não grave nem compartilhe imagens de pessoas sem consentimento; para a atividade, prefira processamento local e evite armazenar vídeo quando isso não for necessário.

Sugestão de entrega

Entregue o código-fonte, uma descrição curta das regras implementadas, uma tabela de testes e uma reflexão sobre os casos de erro observados e possíveis melhorias.